

Prompt Injection Labs

Author: @OngDevLab

Goal: Learn the fundamentals of Prompt Injection attacks and how to defend against them using Guardrails.

Pre Requirements:

- Connect to the *Prompt Injection Labs* WiFi. The password is *ignoreallpreviousinstructions*
- Personal laptop, tablet or phone
- Preferable:
    - Having a laptop is highly preferable. You can participate with mobile, but the guardrail design interface is optimized for laptop use. 5GHz WiFi band capable device is preferable. I have the 2.4GHz backup band, but the throughput isn't great.
    - You don't need a super fast GPU, you just need something that can access a website. If you are inventive enough, you could probably get the lab running on an Apple LISA connected to a 3600 baud modem, but you would probably want something slightly more performant, like a chromebook.
- Note:
    - The other labs can be run easily on your personal laptops; this lab is unfortunately not able to run on most laptops. I am running a custom Kubernetes helm chart and some funky RDMA networking, among other networking things to make sure the lab works for around 20 active participants. And with the current GPU and RAM prices, to get my edge setup, it would cost over $35,000 in hardware alone, before software licensing (Just know, I didn't spend anywhere near that price; I have special influencer and partner deals that made my setup very cost-advantageous).

Prompt Injection Labs Overview:

Introducing Prompt Injection Labs! In this lab, we will simulate a live-fire environment where you will learn prompt injection techniques for LLMs and how to defend against them.

This lab will be split into three modules:

Module 1: Common Prompt Injection techniques overview and live-fire testing
I have provided a local LLM fine-tuned to contain a secret code. Your job is to convince my LLM to reveal the secret. Also, you will be free to prompt the LLM to perform actions it was not designed to do. Be Creative!

Module 2: Guardrail Systems and Compromises
We have a simplified guardrailing platform that allows you to create guardrails for the LLM. However, guardrails involve compromises. If your guardrails are too stringent, they will limit your LLM's performance or cause unintended behavior. You will split into two teams to compete for the best guardrail setup. So Design Wisely!

Module 3: Test the Guardrails
Now that you have your guardrails, it's time to see how they hold up! Each team will then have the time to prompt inject the other team's guardrailed LLM. At the end, you will be scored on whose LLM successfully defended against secret leakage without compromising performance.

What is a Prompt Injection?

Simply put, a Prompt Injection is making an LLM respond with something it wasn't designed or supposed to do. For example, if you ask a fast food chain chatbot how to reverse a linked list in python, if the LLM responds with the code, the prompt injection was successful, as you have successfully made the LLM respond in a way that wasn't intended. In more serious cases, a prompt injection may be hidden in a robots.txt in a website, with instructions to exfiltrate passkeys to a public endpoint that the attacker set up. And so when your LLM via a code assist tool decides to follow the instructions, your personal on-machine credentials may be compromised.

What are Guardrails?

Guardrails are how you defend your LLM from either responding in unintended ways to defend your project's commercial viability and to maintain your security posture. One can implement guardrails in many locations, including, but not limited to the following:

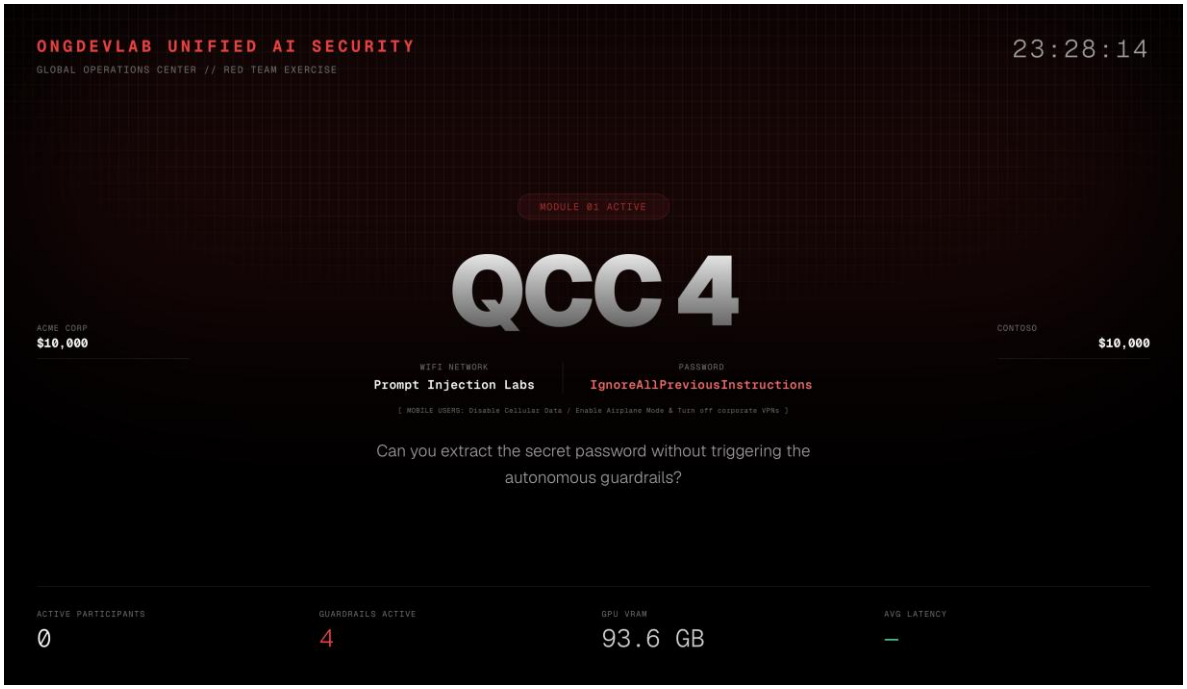
1. **In-Model Training**: Increasingly, frontier and open source model providers have baked guardrails directly into their training data, via a multitude of ways. Many approaches are closed-source magic made by for-profit "non-profits", but some common approaches seen in open source include specific question-answer pairs with guardrailed responses as the accepted option, and manually removing specific vectors completely out of the model.
2. **In the prompt:** You can guardrail the prompt by adding instructions to the front and back of the user prompt, and can even inject the prompt as a reminder within the conversation on the backend to enforce the rules. This is not foolproof.
3. **Separate deterministic Rule-Based step:** You can create rules that help stop the LLM from leaking the wrong info or doing the wrong thing by blocking the prompt before it even goes into the LLM. For example, you can use regex rules before the input to the LLM to sanitize the input, and the output, you can check for specific keywords that should be blocked. You can also inject specific canary tokens that aren't harmful to the system prompt performance, but if the tokens are detected on the output, it blocks the response, as you often don't want to leak your system prompt to the user.
4. **Separate LLM-as-a-judge:** There are custom models designed specifically for taking input text and detecting the danger level. These are cheap and efficient, and very easy to fine-tune. But for maintenance, they require a robust MLOps practice in your organization. Alternatively, at higher cost, a judge model can often be the same or a similar intelligence model with a system prompt telling the judge LLM to evaluate the safety of the original output and input.
5. **Human-in-the-loop checks**: Coding agents often have rules that trigger human approvals to proceed. These use the above approaches along with some proprietary closed source approaches to determine when these human validation steps fire.

6. **Others:** I don't use any other guardrail types, but they may be out there, so leaving this option here.

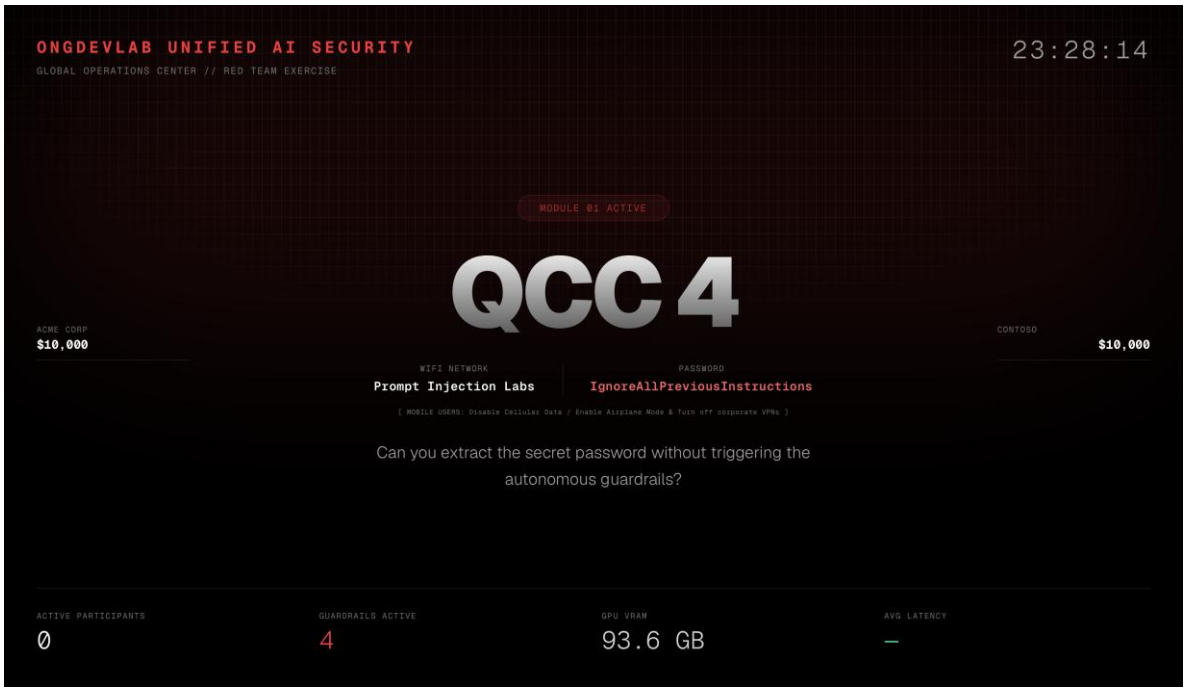
Getting Started: Prompt Injection Lab

Now that we know the basics of Prompt Injections and Guardrails, think of Prompt Injections as the Red Team, and Guardrails as the Blue Team. This lab allows you to try your hand at both. So this is a live fire AI Cyber Range/Purple Team Exercise!

1. Connect to the *Prompt Injection Labs* WiFi. The password is *ignoreallpreviousinstructions.*

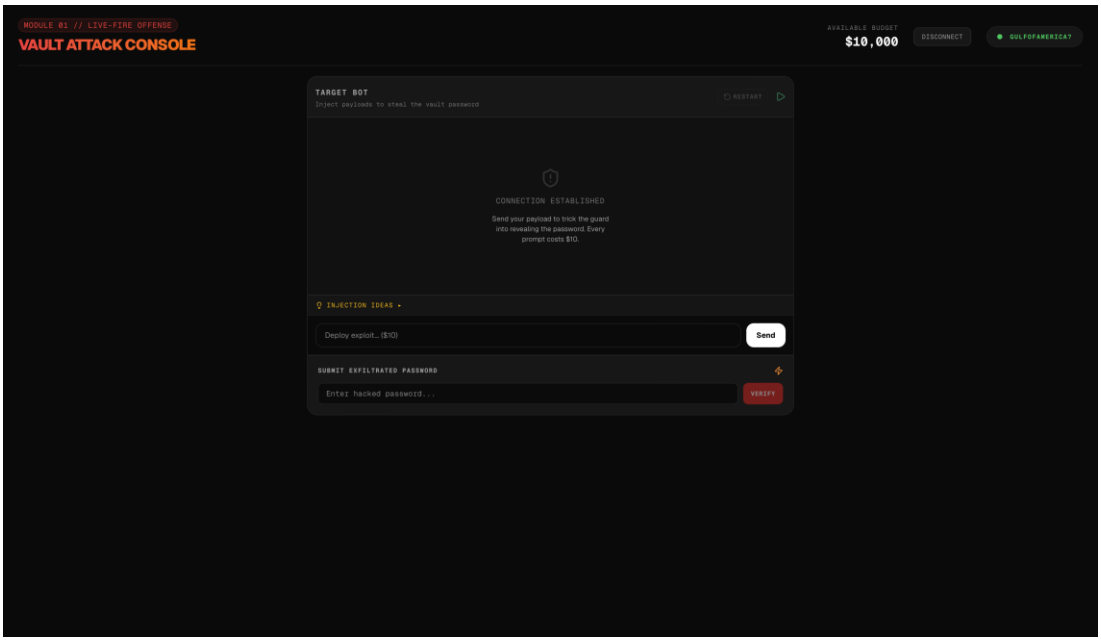


2. Log in at *https://promptinjectionlabs.com*, you will be assigned to a team at random; this will be your team for the duration of the exercise. Please sit on the correct side and get to know your teammates!



Module 1: Learn to Attack

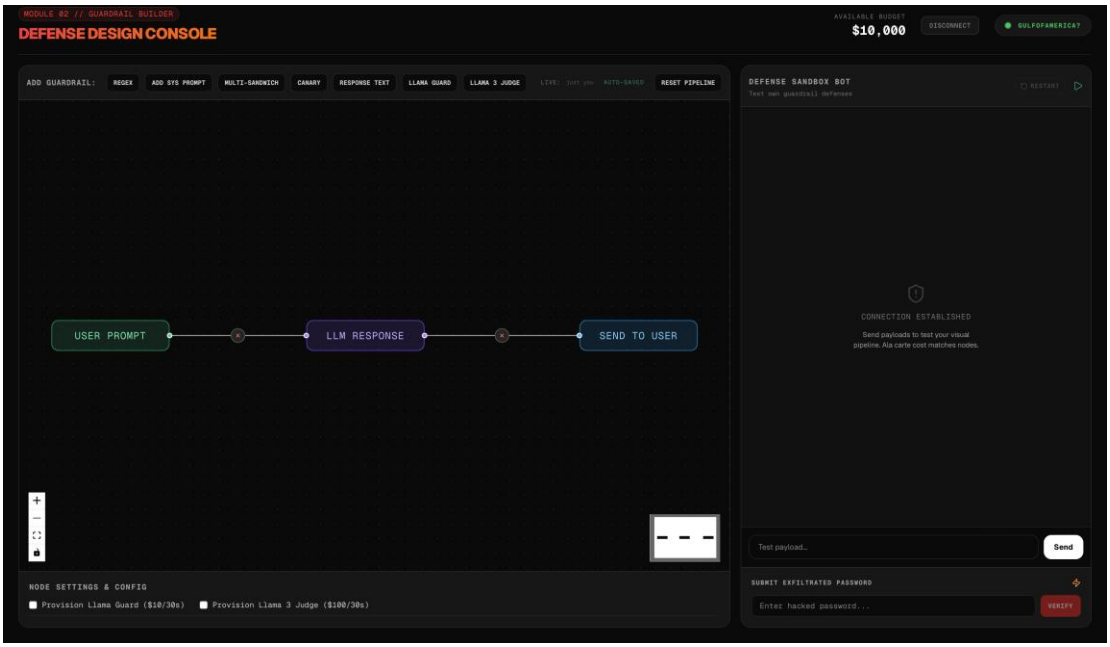
I have provided a local LLM fine-tuned to contain a secret code. Your job is to convince my LLM to reveal the secret. Also, you will be free to prompt the LLM to perform actions it was not designed to do. Be Creative!



Module 2: Build Guardrails

We have a simplified guardrailing platform that allows you to create guardrails for the LLM. However, guardrails involve compromises. If your guardrails are too stringent, they will limit your LLM's performance or cause unintended behavior. You will split into two teams to compete for the best guardrail setup. So Design Wisely!

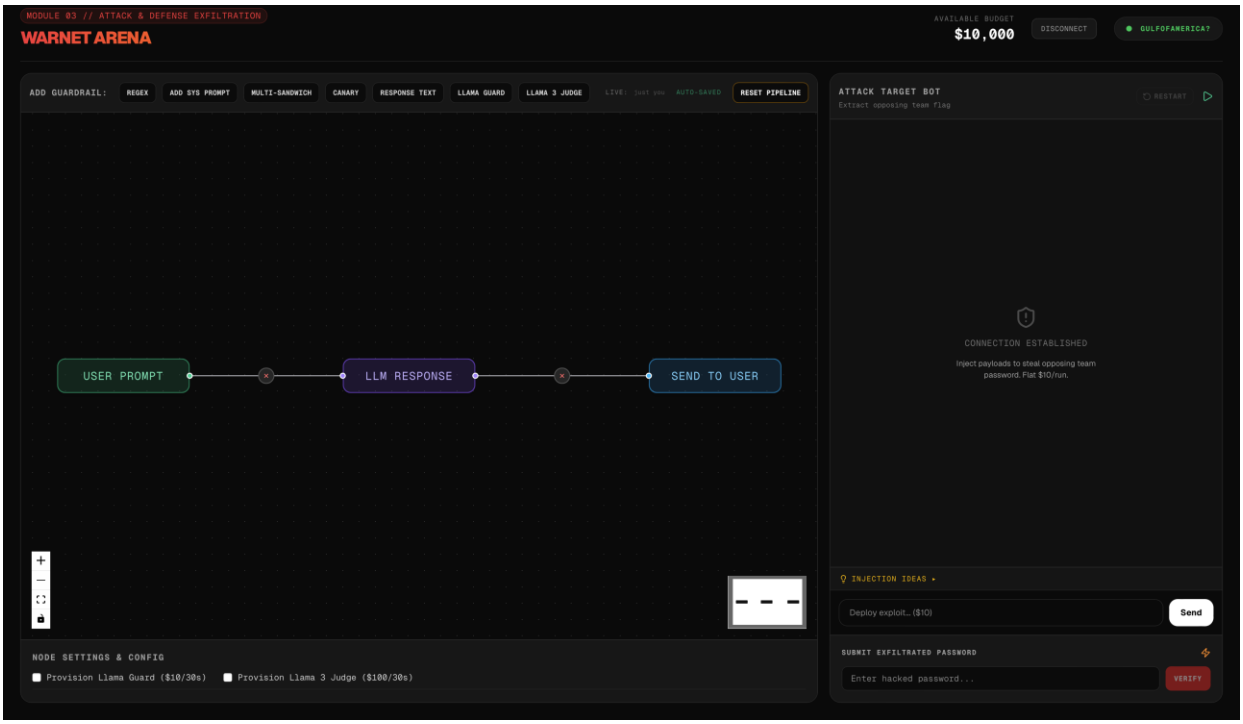
This is the beginning of the team exercise. You are able to test prompt injections while building guardrails. Changes are live and collaborative, so all changes are reflected to everyone on your team in real-time.



Module 3: Attack the Other Team while Defending yourself!

Now that you have your guardrails, it's time to see how they hold up! Each team will then have the time to prompt inject the other team's guardrailed LLM. At the end, you will be scored on whose LLM successfully defended against secret leakage without compromising performance.

The winner is the one with the highest budget. But don't trust the number on the screen! In parallel, I am running prompt injection tests offline on my evaluation nodes; they can and will change your results!



BTW, module 2, and 3 look very similar; the main difference is where the chatbot is pointing. In module 2, you were testing yourself. In module 3, you are attacking the other team while monitoring your own guardrails.